

RAG Evaluation and Diagnostics

pig7selene

September 5, 2026

Abstract

Retrieval-Augmented Generation is an evidence pipeline rather than a single model call. Its evaluation must therefore separate corpus coverage, candidate retrieval, ranking, context construction, answer quality, grounding, citations, and operational cost. This chapter develops the standard ranking metrics used at the retrieval boundary; distinguishes context, answer, and evidence evaluation; and presents controlled ablations, diagnostic traces, regression suites, and online signals. The central principle is that an incorrect final answer is an observation to explain, not a diagnosis of one component.

1 Introduction

Chapter 32 defined the basic Retrieval-Augmented Generation (RAG) path:

query $\rightarrow$ retrieval $\rightarrow$ ranking $\rightarrow$ context construction $\rightarrow$ generation

Every arrow is a possible source of error. A required fact may be absent from the corpus, present but omitted by retrieval, retrieved but ranked below a cutoff, selected but truncated from the final context, available in the context but ignored by the generator, or stated without support or with an incorrect citation. An incorrect answer alone does not identify which of these events occurred. Conversely, a correct answer can arise from parametric knowledge even when the retrieved context is irrelevant. RAG evaluation must therefore make the evidence path observable rather than report only a final answer score.

This chapter closes the RAG and Knowledge Augmentation section. It builds on the corpus, context, and provenance interfaces of Chapter 32; the first-stage and reranking boundaries of Chapters 33–37; and the trace and policy boundaries of Chapter 38. It applies Chapter 20’s cautions about human and model-based judging to an evidence-grounded setting. It does not introduce an evaluation framework tutorial, Agent architecture, or Tool Use.

2 Evaluation Objects and Retrieval Metrics

Let $\mathcal{Q}$ be an evaluation query set. For a query q , let

$$R_{k(q)} = (d_1, \dots, d_k) \quad (1)$$

be the ordered top- k retrieval list, and let $G(q)$ be the set of retrieval units judged relevant under an explicit task definition. Relevance is not an intrinsic property of a passage. A source can mention a query term without supporting the needed claim, while two passages can each be relevant because they supply complementary parts of a multi-hop answer. The corpus revision, retrieval-unit boundaries, filters, and judging guideline are consequently part of every metric definition.

For binary relevance, Recall@ k and Precision@ k are

$$\text{Recall @}k(q) = \frac{|R_{k(q)} \cap G(q)|}{|G(q)|}, \quad \text{Precision @}k(q) = \frac{|R_{k(q)} \cap G(q)|}{k}. \quad (2)$$

Recall@ k asks what fraction of known relevant units reaches the top- k boundary; Precision@ k asks what fraction of that returned prefix is relevant. A binary Hit Rate records whether at least one relevant unit occurs:

$$\text{Hit @}k(q) = 1 \left[R_{k(q)} \cap G(q) \neq \emptyset \right]. \quad (3)$$

Hit Rate is useful when one sufficient passage can answer a question, but it does not distinguish a result at rank one from one at rank k , nor does it reward collecting all complementary evidence. Standard information-retrieval evaluation treats these metrics as properties of ranked lists under relevance judgments, not as universal measurements of answer quality [1].

Ranking-sensitive measures make position explicit. Let $r_{\text{first}(q)}$ be the one-based rank of the first relevant item, with a reciprocal-rank contribution of zero when no relevant item is returned. Then

$$\text{MRR} = \frac{1}{|\mathcal{Q}|} \sum_{q \in \mathcal{Q}} \frac{1}{r_{\text{first}(q)}}. \quad (4)$$

Mean Reciprocal Rank (MRR) rewards placing one useful unit early. It is appropriate when the first supporting result matters most, but it says little about the remaining evidence needed for a complex answer. For graded relevance labels g_i , where larger values indicate a more useful item at rank i , a common discounted cumulative gain is

$$\text{DCG}@k(q) = \sum_{i=1}^k \frac{2^{g_i} - 1}{\log_2(i + 1)}, \quad \text{nDCG}@k(q) = \frac{\text{DCG}@k(q)}{\text{IDCG}@k(q)}. \quad (5)$$

The ideal DCG, IDCG@ k , is the value obtained by ordering the same graded judgments optimally. Normalized Discounted Cumulative Gain (nDCG) therefore rewards highly relevant results near the front while making scores comparable across queries with different relevance-label totals. It is a ranking metric, not proof that an LLM will use the resulting items correctly.

Recall@ k is often a first priority for RAG candidate generation. A later reranker cannot recover evidence that the first stage did not expose, as Chapter 37 emphasized. Yet maximizing recall by increasing k indefinitely is not a solution: more candidates create reranking work, duplicate evidence, and a larger set from which a context constructor can admit noise. The useful operating point depends on the downstream context budget, candidate depth, reranker, and latency target.

Metric	Question it answers	Important limitation
Recall@ k	How much judged relevant evidence reaches a candidate boundary?	Does not reward early placement or assess generator use.
Precision@ k	How much of a retrieved prefix is relevant?	Can penalize a deliberately broad recall-oriented first stage.
Hit Rate@ k	Did at least one relevant unit appear?	Ignores rank and complementary evidence.
MRR	How early is the first relevant item?	Ignores later relevant items after the first hit.
nDCG@ k	Does the ranked prefix place graded relevance near the front?	Requires defensible graded labels and remains a ranking metric.

Table 1: Retrieval metrics measure different properties of a ranked list. Their meaning depends on relevance judgments, cutoff depth, and the role of the list in the later RAG pipeline.

3 Context and Generation Evaluation

The top- k list is not the context that the language model receives. Let $P(q)$ be the set of retrieval units that survive reranking, deduplication, truncation, ordering, and serialization into the final prompt. Let $S(q) \subset G(q)$ be the subset of units sufficient to support the intended answer under the stated task. A useful context-level view is

$$\text{ContextRecall}(q) = \frac{|P(q) \cap S(q)|}{|S(q)|}, \quad \text{ContextPrecision}(q) = \frac{|P(q) \cap G(q)|}{|P(q)|}. \quad (6)$$

These expressions are abstractions, not a claim that all tasks have one complete gold set. Some questions have several valid support paths, and a context can contain a relevant unit that is redundant for a particular answer. Their practical value is to make the difference visible: a retriever may achieve high

candidate Recall@k while the prompt loses the key item through a final token budget, poor ordering, repeated chunks, or a parent-child expansion that dilutes a precise result. Context evaluation must also inspect redundancy, contradictory evidence, source diversity, metadata, and ordering, because each changes what the generator can attend to.

Generation evaluation asks a different set of questions. **Answer correctness** asks whether the answer is factually or task-wise right, preferably through a reference, deterministic verifier, or qualified human judgment. **Answer relevance** asks whether the response actually addresses the user’s information need. **Completeness** asks whether it covers the necessary parts of the answer. These properties are related but nonidentical. A relevant response may be wrong; a correct response may omit a required condition; a complete answer may be unusably verbose for an interactive request.

Reference-based evaluation compares an output with gold answers, labeled relevant passages, or annotated support. It can make a narrow property auditable, but creating high-quality references is expensive and may underrepresent valid paraphrases or alternative evidence paths. Reference-free evaluation instead uses the query, retrieved context, model judges, consistency checks, or structural constraints. It is useful for broad coverage and evolving corpora, but it replaces one missing label with assumptions about the evaluator. RAGAS and ARES exemplify component-aware automated evaluation through categories such as context relevance, answer faithfulness, and answer relevance; their existence does not make any generated score independent ground truth [2], [3].

4 Faithfulness, Grounding, and Citations

Faithfulness or **groundedness** asks whether the answer’s claims are supported by the evidence actually provided to the generator. Let $A(q) = (a_1, \dots, a_M)$ be a decomposition of an answer into evaluable claims, and let $z_j = 1$ when the final context supports claim a_j under a declared support rule. A conceptual claim-support score is

$$\text{Groundedness}(q) = \frac{1}{M} \sum_{j=1}^M z_j. \quad (7)$$

The difficulty is in the support rule. A claim can be directly stated, entailed by several passages together, partially supported, contradicted, or outside the scope of the context. Claim segmentation and entailment judgments can themselves be fallible. An LLM judge can scale the comparison between claims and passages, but it cannot make an unsupported inference reliable merely by expressing a fluent rationale. Chapter 20’s judge-versioning, order-control, and human-audit principles apply directly here.

Groundedness is not answer correctness. A response can be grounded but incomplete because the context lacks a needed fact. It can be correct but unsupported because the model used parametric knowledge after a retrieval miss. It can be relevant but false, or supported by a context that is itself stale or low-authority. These distinctions are diagnostic assets: they identify whether a repair belongs in ingestion, retrieval, context construction, generation, or source policy rather than in generic prompting.

Explicit citations add another interface. **Citation correctness** asks whether a cited source supports the adjacent or associated claim. **Citation completeness** asks whether claims that require support receive suitable citations. **Citation quality** asks whether the cited source is authoritative, current, and appropriate for the claim. A valid-looking source identifier alone establishes none of these. ALCE formalizes end-to-end citation evaluation as a separate dimension alongside answer quality, illustrating why citation coverage and citation entailment should not be collapsed into an answer score [4].

5 Human and Model-Based Evaluation

Human evaluation remains essential when answer usefulness, nuanced correctness, explanatory completeness, or source appropriateness cannot be reduced to a deterministic rule. A useful rubric names the evidence available to annotators, whether they may use external knowledge, what counts as a supported claim, how to score uncertainty, and how to treat a correct but uncited answer. It should record annotator expertise, candidate blinding, response order, inter-annotator disagreement, and the aggregation procedure. Human labels are observations under a rubric, not a universal utility function. LLM-as-a-Judge can evaluate large answer sets cheaply and express natural-language criteria such as relevance or support. It is especially valuable for triage, rapid regression signals, and domains where references are incomplete. Its limitations are the same ones introduced in Chapter 20: position bias, verbosity bias, prompt sensitivity, judge capability limits, self-preference, and correlated errors when the target and judge share a model family. Controlled work on LLM judging documents these risks alongside its practical scalability [5].

A robust design treats judges as calibrated instruments, not oracles. Freeze the model revision, prompt, rubric, examples, temperature, answer order, parser, and aggregation rule. Swap candidate order in pairwise tests; compare a stratified sample with human labels or deterministic checks; and report uncertainty and disagreement. Use different mechanisms for different claims: a verifier for an executable answer, human review for difficult source quality, and a judge for scalable open-ended slices. No single evaluator validates the full RAG path.

6 Evaluation Data and Controlled Ablations

An evaluation set should represent the decisions a real system must make. It needs ordinary user queries and known supporting documents, but also unanswerable requests, ambiguous formulations, multi-hop questions, conflicting evidence, temporally sensitive facts, and queries whose answer depends on a rare identifier or a particular document version. Public benchmark prompts are useful, but the corpus revision, query distribution, and source governance must resemble the operating setting. A synthetic question that perfectly echoes one chunk can make retrieval look easy while failing to test the vocabulary mismatch or ambiguity present in production.

Hard negatives are passages that are lexically or semantically plausible yet do not answer the query. They test whether a retriever or reranker can distinguish an entity’s current policy from an outdated revision, a related product from the named version, or a broad topical summary from an answer-bearing passage. They are particularly valuable because random negatives often make a ranking task artificially easy. A hard negative must still be labeled carefully: an apparently wrong passage may be a legitimate alternative support path under a wider task definition.

Distribution shift is continuous rather than exceptional. New documents change vocabulary and source authority; new user populations change query style; a revised chunking policy changes the retrieval-unit distribution; and a model update can alter query rewriting or answer length. Evaluation is therefore a repeated measurement under versioned corpus and protocol conditions, not a one-time benchmark certificate.

Controlled ablations expose a component’s contribution. Replacing normal retrieval with an **oracle context** that contains known supporting evidence tests the generator and context format without a retrieval miss. Removing a reranker, varying candidate depth or final top- k , changing chunk size, comparing sparse and dense paths, disabling query rewriting, or changing evidence order each tests one declared interface. The controls must keep the corpus, filters, prompts, generation settings, and scoring protocol fixed. Otherwise a reported improvement cannot be assigned to the purported component.

7 Error Attribution and Regression

Diagnostic evaluation begins with the answer trace rather than a single scalar score:

Was the answer wrong? $\rightarrow$ Was sufficient evidence in the corpus?
 $\rightarrow$ Was it retrieved? $\rightarrow$ Was it ranked and included in context?
 $\rightarrow$ Was the evidence interpreted correctly? $\rightarrow$ Was the final claim grounded and cited?

This flow yields a concrete taxonomy. A **corpus failure** means sufficient authoritative evidence was absent or unavailable. An **indexing failure** means an eligible source was not represented correctly. A **query-representation failure** originates in analysis, embedding, rewriting, or routing. A **retrieval failure** omits relevant candidates; a **ranking failure** buries them; a **context-construction failure** removes, truncates, duplicates, or obscures them. A **generation failure** misuses available evidence; a **grounding failure** adds unsupported claims; and a **citation failure** links a claim to an unsupported, incomplete, or unsuitable source. Fine-grained RAG evaluation frameworks similarly motivate separate retrieval and generation diagnostics rather than an undifferentiated final score [6].

Layer	Primary evidence	Representative regression guard
Retrieval	Recall@k, Hit Rate, MRR, nDCG, and source-version coverage	Keep corpus, relevance labels, filters, and candidate depth fixed.
Context	Support coverage, precision, redundancy, ordering, and token budget	Compare selected evidence against retrieved candidates and oracle context.
Generation	Correctness, relevance, completeness, and abstention behavior	Fix prompt, model, decoding settings, and answer parser.
Grounding and citations	Claim support, citation correctness, completeness, and source quality	Audit answer-to-context and claim-to-citation links separately.
Systems	TTFT, latency, throughput, context tokens, and cost	Measure quality and service conditions together under the same workload.

Table 2: A RAG regression suite should preserve evidence at each pipeline boundary. An end-to-end answer metric remains useful, but it cannot replace the component evidence needed to locate a regression.

For a metric M_j and a candidate configuration c relative to a baseline b , record

$$\Delta_j = M_{j(c)} - M_{j(b)}. \quad (8)$$

The vector $(\Delta_j)_j$ is more informative than an aggregate that hides a retrieval, grounding, or latency regression. A release criterion can demand that a new reranker improve nDCG without reducing context support, or that a larger candidate depth improve recall without exceeding a tail-latency budget. This is an application of Chapter 20’s multi-objective regression logic to an observable evidence pipeline.

8 Online and System Evaluation

Offline labels cannot cover every new document, query, or interaction pattern. Online signals may include source-opening or click behavior where it is meaningful, user corrections, query reformulations, abandonment, explicit feedback, task completion, and escalation to a human. These signals are noisy behavioral observations. A user may open a source because the answer was unclear, reformulate after a correct answer for a new purpose, or abandon because the interface is slow rather than because the retrieval was wrong. They should be joined to sampled human review and trace-level diagnostics, not treated as automatic quality labels.

RAG also has system costs. Measure retrieval and reranking latency, generation latency, Time to First Token (TTFT), total response time, context-token count, requests per second, and monetary or computational cost under an explicit workload. Chapters 23, 26, and 29 explain why those quantities have different bottlenecks. Increasing top- k may raise candidate recall, then add reranking work, increase context length, and increase Prefill cost without improving answer correctness. The relevant objective is a quality–latency–cost frontier, not throughput or a retrieval metric in isolation.

9 Failure Modes

The most common evaluation failure is measuring only the final answer or only retrieval. Answer-only evaluation cannot distinguish a missing source from a generator that ignored it; retrieval-only evaluation cannot detect a context builder that truncates the best passage or a model that fabricates a citation. Another failure is relying on one LLM judge without auditing its prompt, order, and distributional biases. A single evaluator can become the unexamined bottleneck of a supposedly multi-component evaluation system.

Unrealistic synthetic queries, weak relevance labels, omitted unanswerable cases, and hard negatives that are not truly negative can make both ranking and answer metrics misleading. Comparing configurations with different corpora, document versions, filters, prompts, decoding settings, or answer parsers confounds a component change with a protocol change. Evaluation leakage is similarly damaging: if benchmark questions, answers, or known support passages are incorporated into the corpus or model-development loop without disclosure, an end-to-end gain may overstate generalization.

Finally, optimizing directly against one proxy creates a Goodhart risk. A system can learn to maximize an LLM judge, retrieve more passages to raise recall, or attach many citations to raise coverage while reducing usefulness, evidence quality, or latency. Chapter 20’s warning applies unchanged: a proxy that becomes an optimization target can stop measuring the property it was chosen to represent. Keep independent audits, protected slices, and full-pipeline traces outside the immediate optimization loop.

10 Implementation Contracts

The **dataset contract** must version the corpus, document and chunk identifiers, relevance and support judgments, query set, query distribution, unanswerable and hard-negative policies, source timestamps, and labeling rubric. It must state whether relevance is binary or graded and distinguish a relevant unit from a minimal answer-supporting unit. The **pipeline contract** must retain every query transformation, retriever and index revision, filter decision, candidate list with ranks and scores, reranker inputs, final context order, prompt template, generator revision, decoding configuration, answer parser, and output citations.

The **scoring contract** must name every metric cutoff, averaging convention, treatment of missing or empty relevance sets, judge or verifier revision, claim-segmentation policy, citation-to-claim alignment rule, and uncertainty calculation. Human evaluation requires annotator qualifications, blinding, order randomization, disagreement handling, and audit sampling. An LLM judge requires the frozen prompt, examples, temperature, model version, response-order policy, parser, repeat-call policy, and calibration checks from Chapter 20.

The **regression contract** should compare a candidate with a baseline under the same corpus revision and workload. It should report component metrics, answer quality, grounding, citation quality, latency distributions, context-token use, throughput, cost, and failure slices for vocabulary mismatch, ambiguity, multi-hop evidence, stale sources, conflicts, unanswerable queries, permissions, and long contexts. Retain representative traces for every material regression. A RAG system is diagnosable only when the evidence that crossed each boundary can be reproduced.

11 Summary

RAG evaluation is necessarily multi-level. Retrieval metrics establish whether candidate ranking exposes relevant evidence; context metrics establish whether answer-supporting material reaches the prompt; generation metrics establish correctness, relevance, and completeness; grounding and citation metrics establish whether the response is supported by the evidence it presents. These measures complement, rather than substitute for, end-to-end evaluation.

The practical discipline is to preserve the entire evidence path. Build versioned evaluation data, use controlled ablations and oracle contexts to isolate components, inspect trace-level failures, and protect quality, groundedness, latency, and cost together in regression tests. This closes the RAG section with its central engineering principle: a reliable system does not merely produce an answer score; it can explain what evidence was available, what evidence was used, and where a failure entered the pipeline.

References

- [1] C. D. Manning, P. Raghavan, and H. Schütze, *Introduction to Information Retrieval*. Cambridge University Press, 2008. doi: 10.1017/CBO9780511809071.
- [2] S. Es, J. James, L. Espinosa-Anke, and S. Schockaert, “RAGAs: Automated Evaluation of Retrieval Augmented Generation,” in *Proceedings of the 18th Conference of the European Chapter of the Association for Computational Linguistics: System Demonstrations*, Association for Computational Linguistics, 2024, pp. 150–158. doi: 10.18653/v1/2024.eacl-demo.16.
- [3] J. Saad-Falcon, O. Khattab, C. Potts, and M. Zaharia, “ARES: An Automated Evaluation Framework for Retrieval-Augmented Generation Systems,” in *Proceedings of the 2024 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies (Volume 1: Long Papers)*, Association for Computational Linguistics, 2024, pp. 338–354. doi: 10.18653/v1/2024.naacl-long.20.
- [4] T. Gao, H. Yen, J. Yu, and D. Chen, “Enabling Large Language Models to Generate Text with Citations,” in *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*, Association for Computational Linguistics, 2023, pp. 6465–6488. doi: 10.18653/v1/2023.emnlp-main.398.
- [5] L. Zheng *et al.*, “Judging LLM-as-a-Judge with MT-Bench and Chatbot Arena,” in *Advances in Neural Information Processing Systems 36*, Curran Associates, Inc., 2023. doi: 10.48550/arXiv.2306.05685.
- [6] D. Ru *et al.*, “RAGChecker: A Fine-grained Framework for Diagnosing Retrieval-Augmented Generation,” in *Advances in Neural Information Processing Systems 37*, Curran Associates, Inc., 2024. doi: 10.52202/079017-0692.